



FRONTEND DESIGN · 2026

Elementos Estruturais Avançados

Formulários, listas e a importância da semântica para acessibilidade.

• Aula 04 · Semestre 1/2

Recapitulando a **Aula Anterior**

Na aula passada, mergulhamos na **Arquitetura em Camadas** da web — a separação entre HTML, CSS e JavaScript. Aprendemos a estrutura base de todo documento HTML, as tags essenciais do `<body>` e os primeiros passos com **HTML semântico**.

Hoje vamos avançar para os elementos que tornam o HTML verdadeiramente **interativo e estruturado**: formulários, listas avançadas e os fundamentos de acessibilidade que todo desenvolvedor precisa conhecer.

Agenda da **Aula**

TEMPO	ATIVIDADE	FOCO
00-10 min	Revisão Relâmpago	Recapitular: camadas da web, estrutura HTML, tags semânticas. O que ficou?
10-25 min	Formulários HTML	A tag <code><form></code> , action, method. Por que formulários são a espinha dorsal da web interativa.
25-45 min	Inputs e Controles	Tipos de input, label, select, textarea, radio, checkbox. Atributos essenciais.
45-60 min	Listas Avançadas	Listas de definição, listas aninhadas, quando usar <code></code> vs <code></code> vs <code><dl></code> .
60-80 min	Semântica e Acessibilidade	ARIA labels, atributos de acessibilidade, validação nativa do navegador.
80-100 min	Exercício Prático	Criar um formulário de cadastro completo com validação e semântica.

A Espinha Dorsal da **Web Interativa**

Formulários são o principal canal de **comunicação entre o usuário e o servidor**. Login, cadastro, busca, checkout, comentários — tudo passa por um `<form>`. Sem formulários, a web

seria apenas leitura.

Um formulário bem construído é **invisível** — o usuário preenche sem pensar. Um formulário mal construído é uma barreira que frustra, confunde e faz o usuário desistir. A diferença está na **semântica e nos detalhes**.



<form>

O **container** de todo formulário. Define para onde os dados serão enviados (**action**) e como (**method** : GET ou POST). Sem **<form>**, os campos existem mas não fazem nada.



action e method

action define a URL de destino dos dados.
method="GET" envia na URL (buscas),
method="POST" envia no corpo da requisição (dados sensíveis, cadastros).



<fieldset> e <legend>

Agrupam campos relacionados visualmente e semanticamente. O **<legend>** dá nome ao grupo — essencial para leitores de tela entenderem a estrutura do formulário.

```
←!— Estrutura básica de um formulário —→
<form action="/cadastro" method="POST">
  <fieldset>
    <legend>Dados Pessoais</legend>

    <label for="nome">Nome completo</label>
    <input type="text" id="nome" name="nome" required>

    <label for="email">E-mail</label>
    <input type="email" id="email" name="email" required>
  </fieldset>

  <button type="submit">Cadastrar</button>
</form>
```

A IMPORTÂNCIA DO <LABEL>

for + id

Conexão explícita

O atributo **for** do **label** aponta para o **id** do **input**. Clicou no **label**? O **input** recebe foco. Essencial para acessibilidade e usabilidade mobile.

label envolvente

Conexão implícita

Envolver o **input** dentro do **<label>** também funciona: **<label>Nome: <input></label>**. Mais simples, mas o **for/id** é mais robusto.

placeholder

Não substitui label

O placeholder é uma dica visual que desaparece ao digitar. Nunca use como substituto do `<label>` — leitores de tela e usabilidade dependem do label real.

04 — INPUTS

Tipos de Input e Controles

O HTML5 trouxe dezenas de tipos de input que **eliminam a necessidade de JavaScript** para validações básicas. Cada tipo ativa comportamentos nativos: teclados específicos em mobile, validação automática e controles visuais do navegador.

#

text, email, password

Os inputs mais comuns. `type="email"` e `type="password"` ativam validação e teclados específicos em mobile. Sempre use o tipo correto — não coloque tudo como `text`.

☑

checkbox e radio

`checkbox` para seleção múltipla, `radio` para seleção única (mesmo `name`). Ambos precisam de `<label>` clicável para boa usabilidade.

📅

date, number, range

Tipos especializados que geram controles nativos do navegador. `date` abre um calendário, `number` permite setas, `range` cria um slider. Zero JavaScript necessário.

💬

<textarea> e <select>

`<textarea>` para textos longos (comentários, mensagens). `<select>` com `<option>` para listas suspensas. Ambos são **tags próprias**, não tipos de input.

VALIDAÇÃO NATIVA DO NAVEGADOR

🛡

required

Marca o campo como obrigatório. O navegador **impede o envio** se estiver vazio — sem JavaScript. Combinado com `type="email"`, valida formato automaticamente.

🕒

minlength e maxlength

Definem limites de caracteres. `minlength="8"` em uma senha garante comprimento mínimo. O navegador valida nativamente antes do envio.

<>

pattern

Aceita uma regex para validação customizada. Ex: `pattern="[0-9]{3}\.[0-9]{3}\.[0-9]{3}-[0-9]{2}"` para CPF. Poderoso, mas use com moderação.

```
←!— Formulário com validação nativa —>
<form>
  <label for="senha">Senha (mínimo 8 caracteres)</label>
  <input type="password" id="senha" minlength="8" required>

  <label for="idade">Idade</label>
  <input type="number" id="idade" min="16" max="120">

  <label>
    <input type="checkbox" required>
    Aceito os termos de uso
  </label>

  <button type="submit">Enviar</button>
</form>
```

Dica: Clique em "Enviar" sem preencher os campos — o navegador mostra mensagens de erro automaticamente, sem nenhum JavaScript. Isso é **validação nativa**.

05 — LISTAS

Listas Avançadas

Listas são muito mais do que bullets e números. Elas são elementos **semânticos poderosos** que organizam informação e comunicam relações entre itens. Menus de navegação? São listas. Breadcrumbs? Listas. Specs de um produto? Listas de definição.

Escolher o tipo certo de lista não é detalhe — é **semântica**. Um leitor de tela anuncia "lista com 5 itens" e permite navegar item a item. Uma sequência de `<div>` não comunica nada disso.



`<ul>` — Lista não ordenada

Para itens **sem ordem específica**: ingredientes, habilidades, links de navegação. Renderiza com marcadores (bullets). A mais usada no HTML.



`<ol>` — Lista ordenada

Para itens onde a **ordem importa**: passos de uma receita, ranking, instruções. Renderiza com números. O atributo `start` permite iniciar de outro número.



`<dl>` — Lista de definição

Para pares de **termo + definição**: glossários, specs, metadados. Usa `<dt>` (termo) e `<dd>` (definição). Subutilizada mas extremamente semântica.

←!— Lista de definição — perfeita para glossários e specs →

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language — linguagem de marcação para estruturar conteúdo na web.
</dd>

  <dt>CSS</dt>
  <dd>Cascading Style Sheets — linguagem para definir a apresentação visual do HTML.</dd>

  <dt>DOM</dt>
  <dd>Document Object Model — representação em árvore do HTML que o JavaScript manipula.
</dd>
</dl>
```

←!— Navegação semântica — a nav é uma lista! →

```
<nav aria-label="Menu principal">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/sobre">Sobre</a></li>
    <li><a href="/contato">Contato</a></li>
  </ul>
</nav>
```

06 — ACESSIBILIDADE

Semântica para Todos

Acessibilidade não é um extra — é parte fundamental do desenvolvimento web. No Brasil, a **Lei Brasileira de Inclusão (13.146/2015)** exige que sites públicos e de serviços sejam acessíveis. Na prática, acessibilidade beneficia **todos os usuários**: desde quem usa leitor de tela até quem está com o braço quebrado usando apenas o teclado.

A boa notícia: se você usar **HTML semântico** corretamente, 80% da acessibilidade já está resolvida. Os atributos ARIA existem para os 20% restantes — quando a tag HTML não é suficiente para comunicar o contexto.



aria-label

Fornece um **nome acessível** quando o texto visível não é suficiente. Ex: um botão com apenas um ícone precisa de `aria-label="Fechar"` para leitores de tela.



aria-describedby

Conecta um campo a uma **descrição adicional**. Ideal para mensagens de erro ou instruções. O leitor de tela anuncia a descrição junto com o campo.



aria-required e aria-invalid

`aria-required="true"` complementa o `required` do HTML. `aria-invalid="true"` sinaliza erro no campo. Juntos, comunicam estado para tecnologias assistivas.



role

Define o **papel** de um elemento quando a tag HTML não é suficiente. Ex: `role="alert"` anuncia mensagens dinâmicas. Use com moderação — a tag semântica correta é sempre preferível.

REGRAS DE OURO PARA FORMULÁRIOS ACESSÍVEIS

Label

Todo input precisa de label

Sem `<label>`, o leitor de tela não sabe o que o campo pede. É o erro de acessibilidade mais comum em formulários — e o mais fácil de corrigir.

Tab

Navegação por teclado

O usuário deve conseguir preencher todo o formulário usando apenas Tab, Espaço e Enter. Se um campo não recebe foco via teclado, algo está errado.

Contraste

Texto legível em qualquer campo

Labels, placeholders e textos de erro precisam ter contraste suficiente (WCAG AA: 4.5:1 para texto normal). Placeholder cinza-claro em fundo branco é um erro clássico.

Erro

Mensagens de erro claras

Não basta pintar o campo de vermelho — diga **o que está errado** e **como corrigir**. "CPF inválido — use o formato 000.000.000-00" é melhor que "Campo inválido".

INACESSÍVEL

Placeholder como label

O placeholder desaparece ao digitar — o usuário esquece o que o campo pedia. Leitores de tela podem não anunciar o placeholder. **Nunca** substitua label por placeholder.

ACESSÍVEL

Label visível + placeholder de exemplo

Label permanente + placeholder como dica:

`<label>CPF</label>` com `placeholder="000.000.000-00"`. Informação sempre visível e acessível.

Mão na Massa

Exercício: formulário de cadastro completo

Crie um formulário de cadastro para um evento fictício (hackathon, conferência, workshop). O formulário deve ser **semântico, acessível e com validação nativa**.

1. Crie um novo arquivo **cadastro.html** no VS Code.
2. Use a estrutura base (! + Tab) e altere o título.
3. Crie um `<form>` com pelo menos 3 `<fieldset>`:

```
<form action="/inscricao" method="POST">

  <fieldset>
    <legend>Dados Pessoais</legend>
    <!-- nome, email, telefone -->
  </fieldset>

  <fieldset>
    <legend>Sobre Você</legend>
    <!-- nível de experiência (radio), linguagens que domina (checkbox) -->
  </fieldset>

  <fieldset>
    <legend>Preferências</legend>
    <!-- turno preferido (select), restrição alimentar (textarea) -->
  </fieldset>

  <button type="submit">Inscrever-se</button>
</form>
```

4. Todo input deve ter `<label>` com `for/id`.
5. Use `required` nos campos obrigatórios.
6. Use os tipos corretos: `email`, `tel`, `number`.
7. Teste a navegação por teclado: `Tab` entre campos, `Espaço` em checkboxes, `Enter` para enviar.
8. **Desafio extra:** adicione `aria-describedby` em campos que precisam de instrução adicional.

Lembre-se: O formulário vai funcionar visualmente simples — estamos focando na **estrutura e semântica**. CSS vem depois. Um formulário bem estruturado é muito mais fácil de estilizar.